POLITECNICO
MILANO 1863

DIPARTIMENTO DI ELETTRONICA
INFORMAZIONE E BIOINGEGNERIA

Internet Of Things

Challenge 2 – PCAP Analysis

Seyedaman Zargari – 11109586
Amirhossein Dibaj – 11109540

Academic Year 2025–2026

Overview

All filters below are Wireshark display filters. Where counting tokens across multiple packets was required, we used `tshark` together with small Python scripts. Malformed packets are ignored throughout, as prescribed by the challenge.

Network reference

Role	Address / port
coap.me (public CoAP server)	134.102.218.18 : UDP 5683
Local CoAP server (A.pcapng)	127.0.0.1 : UDP 5683
Local MQTT broker (A.pcapng)	127.0.0.1 : TCP 1883
Local MQTT broker (B.pcapng)	::1 : TCP 1883
HiveMQ public broker	18.192.151.104 / 35.158.43.69 / 35.158.34.213 : TCP 1883
test.mosquitto.org	5.196.78.28 : TCP 1883

CQ1 — NON Confirmable DELETE requests to coap.me

CQ1a. Find all NON Confirmable DELETE requests directed to the *coap.me* server that received a successful response.

Filter for the requests

```
coap.type==1 and coap.code==4 and ip.dst==134.102.218.18
```

This yields **20** NON DELETE requests from `10.0.2.15` towards `coap.me`. For each of those 20 tokens we looked up the matching response from `134.102.218.18`:

Frame	MID	Token	URI path	Response
44	33264	`7b93...`	`5`	4.05 Method Not Allowed
96	53878	`030b...`	`broken`	4.05
212	63041	`f35b...`	`3`	4.05
342	38052	`2301...`	`weird333`	4.05
2007	57936	`e93d...`	`location1`	4.05
2121	25814	`8b6f...`	`hello`	4.05
2206	1059	`b651...`	`separate`	4.05
2260	32298	`7359...`	`4`	4.05
2912	24716	`e086...`	`seg1`	4.05
3372	38480	`57f9...`	`secret`	4.05
3597	32242	`7801...`	`seg1`	4.05
4982	30212	`252e...`	`path`	4.05
5379	11787	`21a5...`	`3`	4.05
7807	22836	`7f09...`	`4`	4.05
8442	63788	`07f2...`	`5`	4.05
8447	47220	`1e60...`	`3`	4.05
8524	62535	`7370...`	`seg1`	4.05
9250	49932	`452a...`	`secret`	4.05
11075	**30800**	`880f30d8b543d6f7`	`validate`	**2.02 Deleted** (frame 11078)
11923	21028	`7591...`	`3`	4.05

Only one of the twenty DELETE requests receives a *successful* 2.xx response: the one with MID **30800** targeting the resource `/validate`.

Answer: CQ1a = **MID 30800** (unique NON DELETE to `/validate` that got a 2.02 *Deleted* response)

CQ1b. Of the requests found in CQ1a, how many actually obtained the desired outcome (i.e. the resource was really removed)?

Although the server answered 2.02 *Deleted*, we verified whether the resource was genuinely gone by searching for *any* subsequent access to `/validate`:

```
coap and ip.dst==134.102.218.18 and coap.opt.uri_path=="validate"
```

Frame	Type/Code	MID	Method	Comment
11075	NON DELETE	30800	DELETE	the request from CQ1a
11078	NON 2.02	26010	(resp)	server says "Deleted"
11363	NON GET	5096	GET	client re-reads `/validate` after the DELETE
11364	NON 2.05 Content	41395	(resp)	**resource still returns data!**

The GET at frame 11363 gets a 2.05 *Content* response *after* the DELETE was acknowledged. The resource still exists on the public `coap.me` server: the 2.02 *Deleted* was returned, but the actual state did not change (coap.me is a test server that always answers DELETE with 2.02 even when the resource is not truly removable). Therefore the desired outcome was *not* achieved.

Answer: CQ1b = **0**

CQ2 — Unique unsuccessful CON POST vs. CON PUT on local CoAP resources

For every resource *belonging to the local CoAP server* we count the number of *unique* CON POSTs (X) and CON PUTs (Y) that received an unsuccessful response (CoAP code ≥ 128, i.e. 4.xx/5.xx). Uniqueness is defined by the tuple (URI path, URI query, payload), so exact retransmissions collapse into a single request. Only resources that the server advertises via `/.well-known/core` are considered "resources of the local server" — requests directed at non-existent paths return 4.04 Not Found but do not belong to any server resource.

Resources advertised by the server (`/.well-known/core`)

```
</dining_room>;ct=0;if="if1";rt="rt1",
</dining_room/door>;ct=0,
</dining_room/temperature>;obs,
</hello_post>;ct=0,
</hello_world>;ct=0,
</living_room/temperature>;obs,
</test>;obs
```

Filters

```
(* CON POST to local server *)
coap.type==0 and coap.code==2 and ip.dst==127.0.0.1

(* CON PUT to local server *)
coap.type==0 and coap.code==3 and ip.dst==127.0.0.1

(* Unsuccessful response from local server *)
coap and ip.src==127.0.0.1 and coap.code>=128
```

Token-matching result

Resource (URI path)	Unique unsucc. POST (X)	Unique unsucc. PUT (Y)	$X = Y$
`dining_room`	1	1	✓
`dining_room/door`	2	1	
`dining_room/temperature`	2	1	
`hello_post`	3	0	
`hello_world`	2	0	
`living_room/temperature`	1	0	
`test`	1	0	

Only `/dining_room` has $X = Y > 0$ among the resources advertised by the server.

Answer: CQ2 = **1** (`/dining_room`: one unique unsuccessful POST and one unique unsuccessful PUT)

CQ3 — Observe notifications on `/dining_room/temperature`

CQ3a. How many *separate* observe notifications were sent for `/dining_room/temperature`, ignoring retransmissions?

The observe registration is the GET at frame 4343 (MID 10588, token `89:b9`). Every subsequent notification reuses the same token; retransmissions share the same MID as the original notification.

Filter

```
coap.code==69 and coap.token==89:b9
```

#	Frame	MID	observe	Notes
–	4344	10588	1	*piggybacked* in the ACK to the registration GET (NOT a separate no
1	4620	35	1	CON, first separate notification
2	5098	38	2	CON
3	5623	43	3	CON
4	8509	45	4	CON
5	9414	49	5	CON
6	9866	54	6	CON (+ 4 retransmits at 9937, 10001, 10193 — same MID 54)
7	10573	62	7	CON (+ 1 retransmit at 10631)
8	11220	74	8	CON
9	11917	87	9	CON
10	12782	90	10	CON

After collapsing every CON retransmission (matched by identical MID within the same observe sequence) and excluding the piggybacked response to the registration, there are exactly **10** distinct notifications (observe sequence numbers 1 → 10).

Answer: CQ3a = **10**

CQ3b. How many of these notifications are useless (a waste of traffic)?

For every CON notification we checked whether the client ACK'd it with a matching MID:

```
coap.type==2 and coap.mid==<MID>
```

Seq.	Frame	MID	ACKed by client?	Useless?
1	4620	35	Yes (frame 4624)	No
2	5098	38	Yes (frame 5099)	No
3	5623	43	Yes (frame 5629)	No
4	8509	45	Yes (frame 8510)	No
5	9414	49	Yes (frame 9415)	No
6	9866	54	No (4 retransmits, no ACK)	**Yes**
7	10573	62	No (1 retransmit, no ACK)	**Yes**
8	11220	74	No (no ACK)	**Yes**
9	11917	87	No (no ACK)	**Yes**
10	12782	90	No (no ACK)	**Yes**

From observe = 6 onwards the client stopped responding. Per RFC 7641, once max retransmissions is exceeded the server should deregister the observer; instead it kept sending observe = 7, 8, 9, 10 which the client never received. All **5** unacknowledged notifications (observe = 6. . . 10) are therefore wasted traffic.

Answer: CQ3b = **5**

CQ4 — MQTT-SN messages received from local broker on port 1885

The MQTT-SN dissector was registered on UDP port 1885 in Wireshark (*Edit* → *Preferences* → *Protocols* → *MQTT-SN*) before running the filters.

Filter — traffic TO port 1885 (clients → alleged broker)

```
udp.dstport==1885
```

Filter — traffic FROM port 1885 (broker → clients)

```
udp.srcport==1885
```

The destination filter returns **438** client datagrams towards UDP 1885, each one immediately followed by an ICMP *Destination unreachable (Port unreachable)*. The source filter returns **0 packets** — nothing listens on that port. Hence no client ever receives an MQTT-SN message from a broker on port 1885.

Answer: CQ4 = **0**

CQ5 — Will messages received via a wildcard subscription (B.pcapng)

Filter — CONNECT packets with Will flag

```
mqtt.msgtype==1 and mqtt.conflag.willflag==1
```

Four clients register a will:

Frame	Broker	Will topic	Will payload
4	::1 local	`university/department12/room1/temperature`	"error: a VIP Client
196	test.mosquitto.org	`metaverse/room2/floor4`	"error: peuivdql"
352	test.mosquitto.org	`hospital/facility3/area3`	"error: xlzaqptd"
557	test.mosquitto.org	`metaverse/room2/room2`	"error: zjzwrcdp"

A will is actually delivered only when the broker (1) detects an abnormal disconnect and (2) has at least one subscriber whose subscription filter matches the will topic. We hunted for PUBLISH packets originating from any broker (`tcp.srcport==1883`) whose payload equals a recorded will message. Only the local-broker will was actually delivered in this capture:

Filter — will delivery (local broker)

```
mqtt.msgtype==3 and tcp.srcport==1883 and
mqtt.topic=="university/department12/room1/temperature" and
mqtt.msg==6572726f723a20612056495020436c69656e74206a7573742064696564
```

Four deliveries were observed (frames 6560, 6562, 6564, 6566, going to client ports 39551, 53557, 51743, 41789). For each of these clients we listed their SUBSCRIBE history (`mqtt.msgtype==8 and tcp.srcport==<port>`):

Subscriber (port)	Subscription that matched	Wildcard?
`::1:39551`	`university/department12/room1/temperature`	No (exact match)
`::1:53557`	`university/department12/room1/temperature`	No (exact match)
`::1:51743`	`university/#` (among other filters)	**Yes**
`::1:41789`	`university/department12/room1/temperature`	No (exact match)

Only the subscriber on port 51743 reached the will message via a wildcard subscription (`university/#`); the other three subscribed with the exact will topic.

Answer: CQ5 = **1** (subscriber `::1:51743` via filter `university/#`)

CQ6 — Clients erasing retained values on HiveMQ

CQ6a. An "erase retained" message is a PUBLISH with `retain = 1` and an *empty* payload. We isolated these towards any of the three HiveMQ IPs returned by DNS (18.192.151.104, 35.158.43.69, 35.158.34.213):

```
mqtt.msgtype==3 and mqtt.retain==1 and
(ip.dst==18.192.151.104 or ip.dst==35.158.43.69 or ip.dst==35.158.34.213)
```

Empty payloads are those where `mqtt.msg` is `<MISSING>` (i.e. the TShark field is not present because the remaining length covers only the topic and, for QoS 1, the packet-id). A short Python script then walked every such erase event and resolved the originating TCP flow back to the `mqtt.clientid` of its CONNECT packet.

Result — 73 erase packets, coming from 5 distinct clients:

Client ID	Length (bytes)	> 7 bytes?
atqeblw	7	No
pcbtjyj	7	No
pohujuv	7	No
giuxfijwus	10	**Yes**
rolsmcwhshlbc	13	**Yes**

Answer: CQ6a = **5** distinct clients

CQ6b. Of the 5 clients, only `giuxfijwus` (10 bytes) and `rolsmcwhshlbc` (13 bytes) have a client-ID strictly longer than 7 bytes. The other three are exactly 7 bytes, which does *not* satisfy the strict inequality.

Answer: CQ6b = **2**

CQ7 — Subscribe requests to the local broker with ≥ 2 wildcards

Filter — SUBSCRIBE to local broker (`::1:1883`)

```
mqtt.msgtype==8 and ipv6.dst==::1 and tcp.dstport==1883
```

We counted the wildcard characters (`+` and `#`) in every topic filter from every SUBSCRIBE message. Those containing at least two are listed below:

Frame	Topic filter	# wildcards
226	university/+/+/deposit	2
363	hospital/+/+/light	2
460	metaverse/+/room1/#	2
473	house/department3/+/+	2
700	metaverse/facility0/+/+	2
1177	hospital/fxlugs/+/#	2
1915	metaverse/+/+/hydraulic_valve	2
2360	factory/+/#	2
2994	house/+/room1/#	2

Answer: CQ7 = **9**

CQ8 — Topic-depth distribution of PUBLISH to the local broker

For each capture we isolated PUBLISH messages whose destination is the local broker (`127.0.0.1:1883` in A.pcapng, `::1:1883` in B.pcapng), computed the topic depth as *(number of / characters) + 1*, and built the grouped histogram below.

Filters

```
(* A.pcapng *)
mqtt.msgtype==3 and ip.dst==127.0.0.1 and tcp.dstport==1883

(* B.pcapng *)
mqtt.msgtype==3 and ipv6.dst==::1 and tcp.dstport==1883
```

Plotting script (`cq8_plot.py`)

The script calls `tshark` twice (once per capture, with the right filter for that capture's local broker), maps each topic to its depth, and draws a grouped bar chart.

```python
import subprocess
from collections import Counter
import matplotlib.pyplot as plt
import numpy as np


def topic_depths(pcap, broker_filter):
    """Return the topic depth of every PUBLISH to the local broker."""
    out = subprocess.run(
        ["tshark", "-r", pcap,
         "-Y", f"mqtt.msgtype==3 and {broker_filter}",
         "-T", "fields", "-e", "mqtt.topic"],
        capture_output=True, text=True).stdout
    return [t.count("/") + 1 for t in out.splitlines() if t.strip()]


a = Counter(topic_depths("A.pcapng", "ip.dst==127.0.0.1 and tcp.dstport==1883"))
b = Counter(topic_depths("B.pcapng", "ipv6.dst==::1 and tcp.dstport==1883"))

print(f"A.pcapng: total={sum(a.values())}, per-depth={dict(sorted(a.items()))}")
print(f"B.pcapng: total={sum(b.values())}, per-depth={dict(sorted(b.items()))}")

depths = sorted(set(a) | set(b))
x, w = np.arange(len(depths)), 0.38
fig, ax = plt.subplots(figsize=(9, 5.5))
ba = ax.bar(x - w/2, [a[d] for d in depths], w, label="A.pcapng", color="#1f77b4")
bb = ax.bar(x + w/2, [b[d] for d in depths], w, label="B.pcapng", color="#ff7f0e")
ax.bar_label(ba, fmt="%d"); ax.bar_label(bb, fmt="%d")
ax.set_xticks(x, depths)
ax.set_xlabel("Number of topic layers")
ax.set_ylabel("Number of PUBLISH messages")
ax.set_title("Distribution of MQTT topic depth for local broker")
ax.legend(); ax.grid(axis="y", linestyle="--", alpha=0.5); ax.set_axisbelow(True)
plt.tight_layout()
plt.savefig("cq8_histogram.png", dpi=150, bbox_inches="tight")
```

Script output

```
A.pcapng: total=439, per-depth={2: 154, 3: 131, 4: 154}
B.pcapng: total=534, per-depth={1: 21, 2: 172, 3: 162, 4: 179}
```

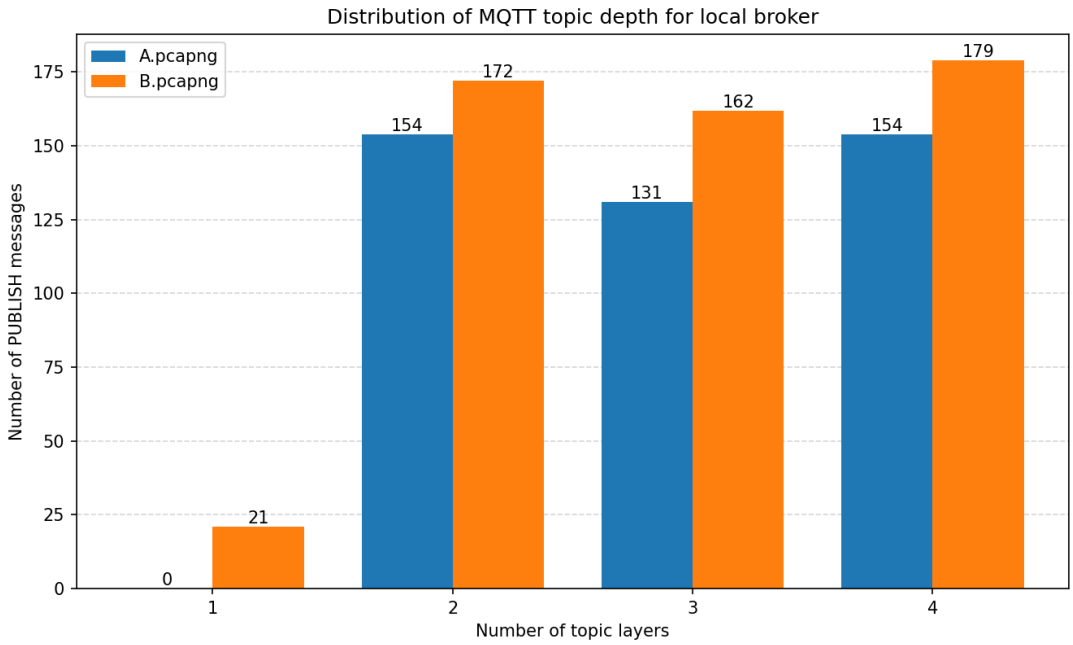

Figure 1: Distribution of MQTT topic depth for local-broker PUBLISH messages.

Depth (layers)	A.pcapng	B.pcapng
1	0	21
2	154	172
3	131	162
4	154	179
Total	**439**	**534**

Answer: CQ8a (total PUBLISH to local broker in A.pcapng) = **439**
CQ8b (total PUBLISH to local broker in B.pcapng) = **534**

Observations. A.pcapng's local broker is used only with 2-, 3-, and 4-layer topics (smart-home layout: `<room>/<device>[/<subresource>]`). B.pcapng additionally exhibits 21 single-layer topics (`$SYS`-style metrics) and is overall more populated at every depth, reflecting a broader experimental scenario.